

Лабораторная работа №2

Содержание

1. Задание 	3
1.1. Дополнительные задания 	4
2. Шаг 1. Создание проекта 	5
3. Основные элементы Windows Forms	7
4. Шаг 2. Создание формы 	9
5. Шаг 3. Добавление классов 	15
6. Шаг 4. Реализация добавления данных в таблицу 	16
7. Шаг 5. Реализация фильтрации (поиска) данных 	21
8. Шаг 6. Сериализация, десериализация классов. Сохранение в XML 	22
9. Дополнительная информация ?	30
10. Стилизация интерфейса 	32
11. Форматирование данных 	35
12. Интерфейс приложения	37

List of Listings

5.1	Пример структуры класса	15
6.1	Метод добавления данных в форму	17
6.2	Метод обновления DataGridView	18
6.3	Метод удаления строки из DataGridView	19
6.4	Добавление данных в столбцы DataGridView без автогенерации	19
6.5	Метод скрытия панелей	20
7.1	Метод фильтрации (поиска) данных в таблице	21
8.1	Пример сериализации данных с помощью DataTable	23
8.2	Пример сериализации данных с помощью DataTable	25
8.3	Пример метода десериализации ¹	26
8.4	Метод десериализации с параметром	27
8.5	Объект OpenFileDialog	27
8.6	Выбор файла и проверка на корректную таблицу	28
8.7	Сериализация XML	28
8.8	Связь событий и кнопок с помощью Tag	29
10.1	Функции реализации перемещения формы	32
10.2	Обработчики движения мыши	33
10.3	Изменение цвета Label внутри GroupBox	34
11.1	Метод форматирования строки со стоимостью	35
11.2	Вызов метода форматирования	36

Лабораторная работа №2

1. Задание ²

1. Реализовать оконное приложение на Windows Forms.
2. Оконное приложение должно позволять создавать объекты, отображать список созданных объектов в табличной форме и выполнять поиск и удаление.
3. Реализовать сериализацию и десериализацию классов в приложении.
4. Сохранить наследование в классах (как в лабораторной работе №1).
5. Сделать возможность добавления нового объекта для каждого класса.
6. Интерфейс приложения должен быть дополнен кнопками для сохранения и загрузки классов из XML файлов.
7. Операции сохранения и загрузки XML файлов должны быть выполнены с использованием методов `async` и `await`.
8. Поиск данных должен выполняться для каждого класса отдельно.
9. Реализовать удаление данных (сохраняя верный порядок строк).

²Варианты заданий в соответствии с вариантом лабораторной работы № 1.

1.1. Дополнительные задания ³

★ Возможность изменения темы интерфейса

- Добавить выбор цветовой схемы (светлая или тёмная тема).
- Использовать Flat стиль кнопок для современного вида.

🔍 Улучшенный поиск

- Реализовать фильтрацию данных в DataGridView в реальном времени (например, при вводе текста в TextBox).
- Поиск с подсветкой найденных результатов.

☰ Контекстное меню

- Добавить правый клик на строку в DataGridView, чтобы появлялось меню с опциями (Редактировать или Удалить).

💾 Автосохранение

- Автоматически сохранять данные в XML при закрытии программы.
- Загружать последние данные при запуске.

📁 Drag & Drop

- Сделать так, чтобы XML-файлы можно было просто перетаскивать в окно приложения для загрузки.

📤 Экспорт в другие форматы

- Позволить сохранять данные не только в XML, но и в JSON.

📊 Статистика

- Добавить внизу статус-бар с инфо: "Объектов в базе: X".

⚠️ Логирование ошибок

- Если что-то пошло не так (ошибки при загрузке XML), выводить понятное сообщение пользователю.

📋 Генерация тестовых данных

- Добавить кнопку «Сгенерировать тестовые данные» для удобства тестирования.

🗂️ Горячие клавиши

- Например, Ctrl + S для сохранения, Ctrl + O для загрузки, Esc для очистки формы.

2. Шаг 1. Создание проекта ✨

- «Создать проект»
- «Windows Forms (Майкрософт)» (Рисунок 2.1)

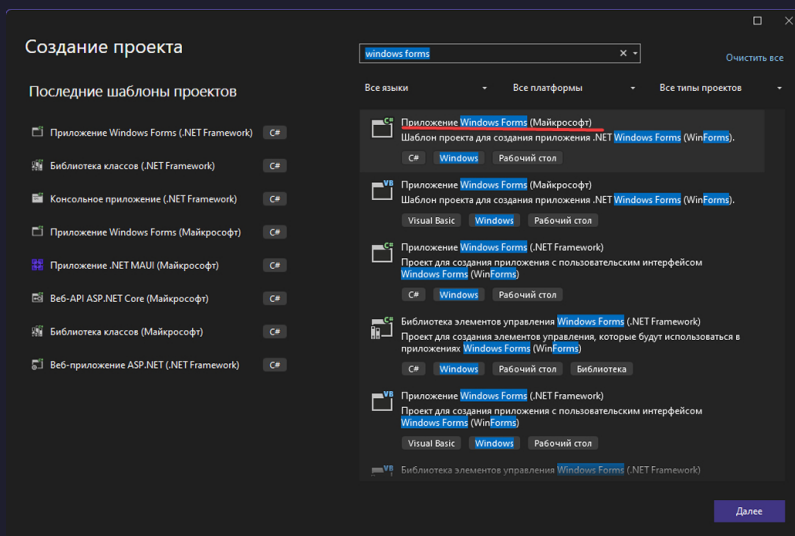


Рисунок 2.1 – Обозначения разделов

- Указываете название формы (содержащее номер лабораторной работы).
- Выбираете последнюю версию платформы из доступных (в примере .NET 8.0) (Рисунок 2.2).

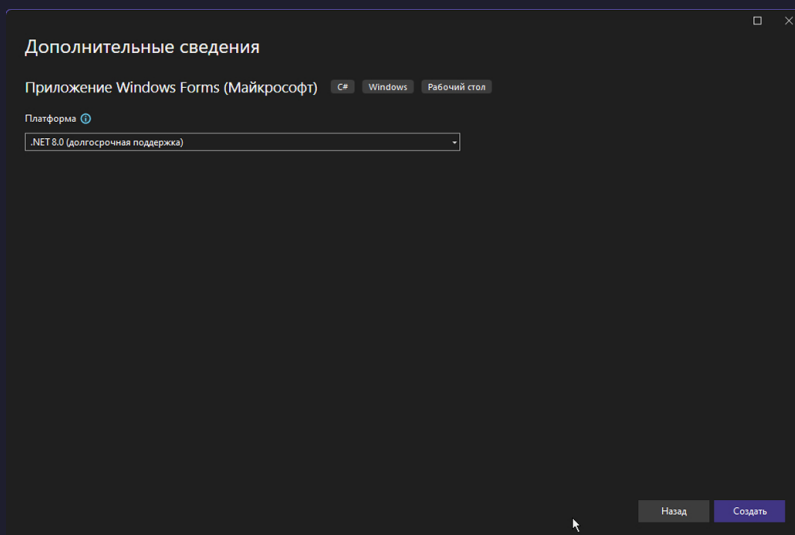


Рисунок 2.2 – Настройки проекта

³Дополнительные задания выполняются по желанию и не подлежат проверке на зачёте.

Перед вами откроется проект с формой по умолчанию. Слева будет «Панель элементов» (если ее нет, перейти в «Вид» → «Панель элементов» / Ctrl + Alt + X) (Рисунок 2.3).

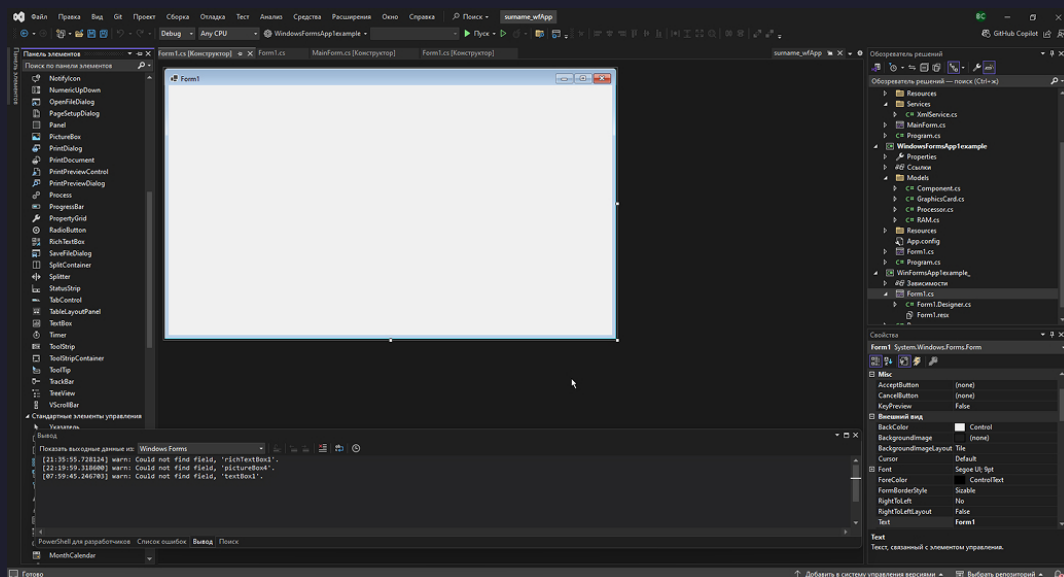


Рисунок 2.3 – Окно проекта

3. Основные элементы Windows Forms

Панель элементов содержит множество элементов для работы с формами. Можно выделить наиболее часто используемые:

- **Form:** Основное окно приложения. Служит контейнером для других элементов управления, предоставляет базовую функциональность для создания пользовательского интерфейса (размещение, масштабирование, обработка событий формы).
- **Button:** Кнопка, по нажатию на которую происходит выполнение заданного кода. Используется для инициирования действий (например, отправка данных, переход на другую форму).
- **Label:** Элемент для отображения текста. Предназначен для вывода информационных сообщений, описания других элементов и инструкций. Не поддерживает ввод данных.
- **TextBox:** Поле для ввода и отображения однострочного текста. Используется для сбора пользовательского ввода, поиска, ввода паролей (с установкой `PasswordChar`) и т.д.
- **RichTextBox:** Расширенная версия `TextBox`, позволяющая форматировать текст (изменять шрифты, цвета, стили). Полезна для редактирования многострочного форматированного текста.
- **CheckBox:** Флажок, позволяющий выбирать или снимать выбор (логическое значение `true` или `false`). Применяется в формах для подтверждения, включения настроек и т.п.
- **ListBox:** Список, позволяющий отображать набор элементов, из которых пользователь может выбрать один или несколько. Простой способ представления данных в виде списка.
- **ComboBox:** Комбинированный элемент, который сочетает в себе поле для ввода и выпадающий список. Удобен для выбора одного значения из набора, позволяя пользователю как выбрать из списка, так и ввести значение вручную.
- **ListView:** Многофункциональный список с поддержкой различных видов отображения (иконки, подробности, плитка). Позволяет отображать элементы с подэлементами, а также использовать возможности сортировки и фильтрации (но редактирование ограничено).
- **DataGridView:** Табличное представление данных с поддержкой привязки данных, сортировки, фильтрации и встроенного редактирования ячеек.
- **PictureBox:** Элемент для отображения изображений. Поддерживает различные форматы, позволяет масштабировать или изменять размер изображения в соответствии с настройками.

- **Panel:** Контейнер для других элементов управления, позволяющий группировать связанные элементы и управлять их компоновкой. Может использоваться для создания сложных макетов или динамического отображения или скрытия групп элементов.
- **GroupBox:** Контейнер с рамкой и заголовком, предназначенный для группировки элементов управления, которые логически связаны между собой. Улучшает визуальную организацию интерфейса.
- **MenuStrip:** Панель меню, располагаемая обычно в верхней части формы. Позволяет создавать выпадающие меню (например, Файл, Правка, Вид и т.д.) для доступа к командам приложения.
- **ToolStrip:** Панель инструментов, содержащая кнопки, текстовые поля, комбобоксы и другие элементы, предназначенные для быстрого доступа к функциям приложения.
- **StatusStrip:** Элемент, располагаемый в нижней части формы, для отображения статусной информации (например, индикаторы, сообщения, время работы).
- **TabControl:** Элемент, позволяющий создавать вкладки для организации контента в пределах одного окна. Удобен для разделения функциональности на логически связанные секции.

4. Шаг 2. Создание формы

Для примера будет разработана форма, содержащая следующие элементы:

1. `DataGridView` – для добавления элементов и отображения их в табличном виде.
2. `Button` – для добавления элементов, удаления и поиска.
3. `TextBox` – для ввода необходимых данных при добавлении элемента или поиске.
4. `PictureBox` – для добавления изображения.
5. `Panel` – для группировки элементов, создания «вкладок».
6. `Label` – для подписи элементов.

Необходимо разместить соответствующие элементы на форме (пока что схематично). Пример формы изображен на рисунке 4.1.

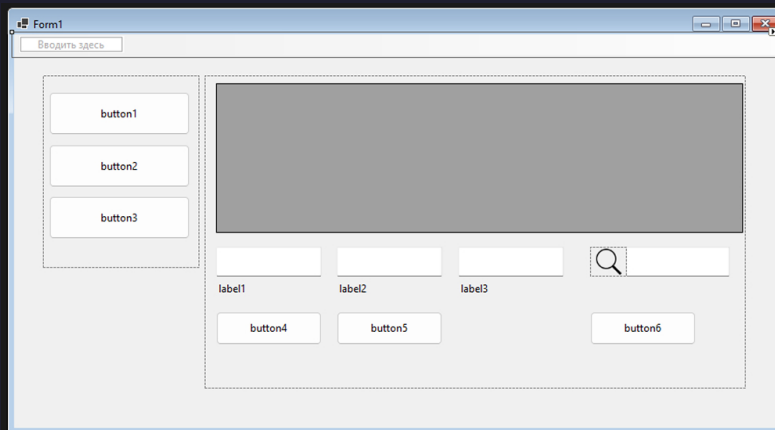


Рисунок 4.1 – Пример оформления формы

Возвращаясь к настройке внешнего вида формы, необходимо рассмотреть свойства элементов формы. В правом нижнем углу в проекте можно увидеть окно свойств элемента. Рассмотрим основные свойства, настройка которых потребуется для создания интуитивного и понятного интерфейса, а также изменения внешнего вида элементов. В верхней части панели есть кнопки для сортировки и перехода по разделам (свойства, эффекты). Для того, чтобы быстрее найти нужное свойство, можно выбрать сортировку «по категориям». Для того чтобы изменить название элемента в коде (при обращении к нему, вызове метода), используется свойство (`Name`). Его можно найти в разделе «Design» («Разработка») (Рисунок 4.2).

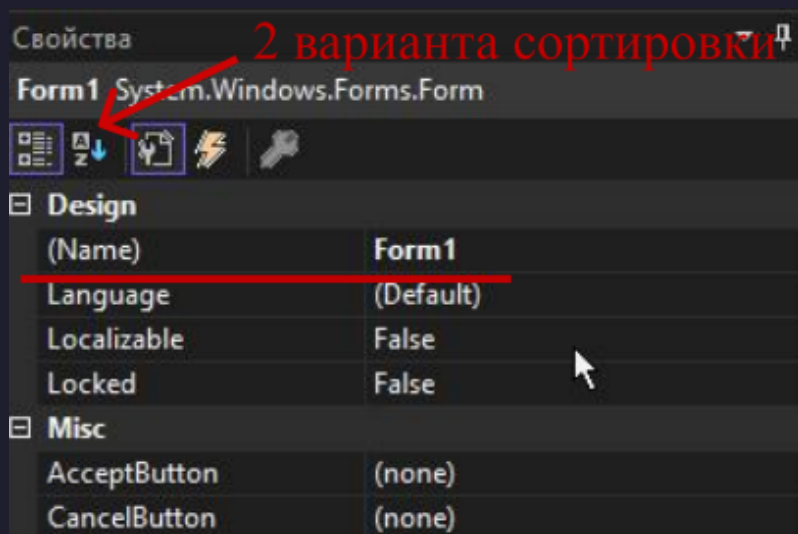


Рисунок 4.2 – Свойство Name

Для того чтобы изменить текст, отображаемый на элементе, необходимо указать новое название в свойстве «Text» в разделе «Внешний вид» (Рисунок 4.3).

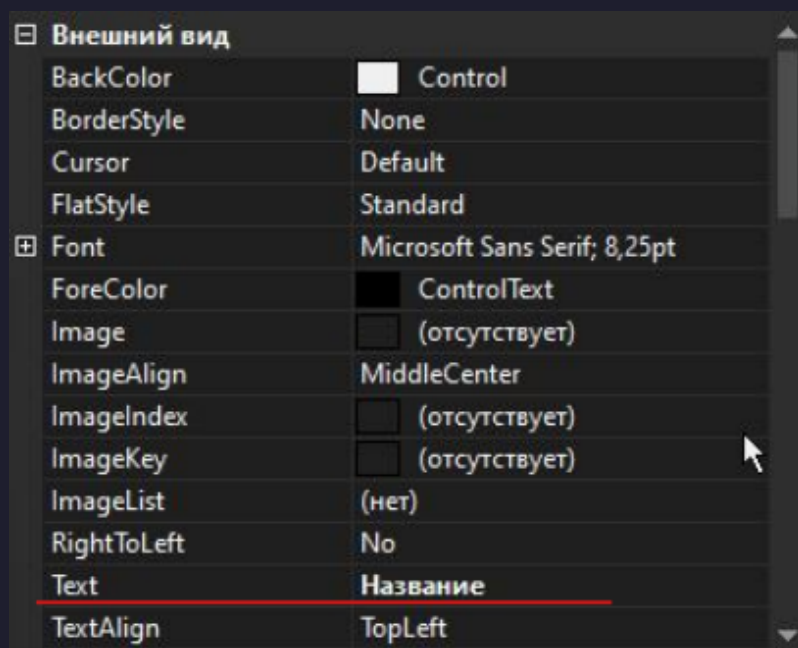


Рисунок 4.3 – Свойство text

Также можно использовать свойство PlaceholderText, чтобы изменить текст, отображаемый на элементе. Свойство находится в разделе «Misc» (Рисунок 4.4). Данное свойство доступно при создании проекта Майкрософт.

³Подробно про свойства можно почитать по ссылке: <https://metanit.com/sharp/windowsforms/2.2.php>

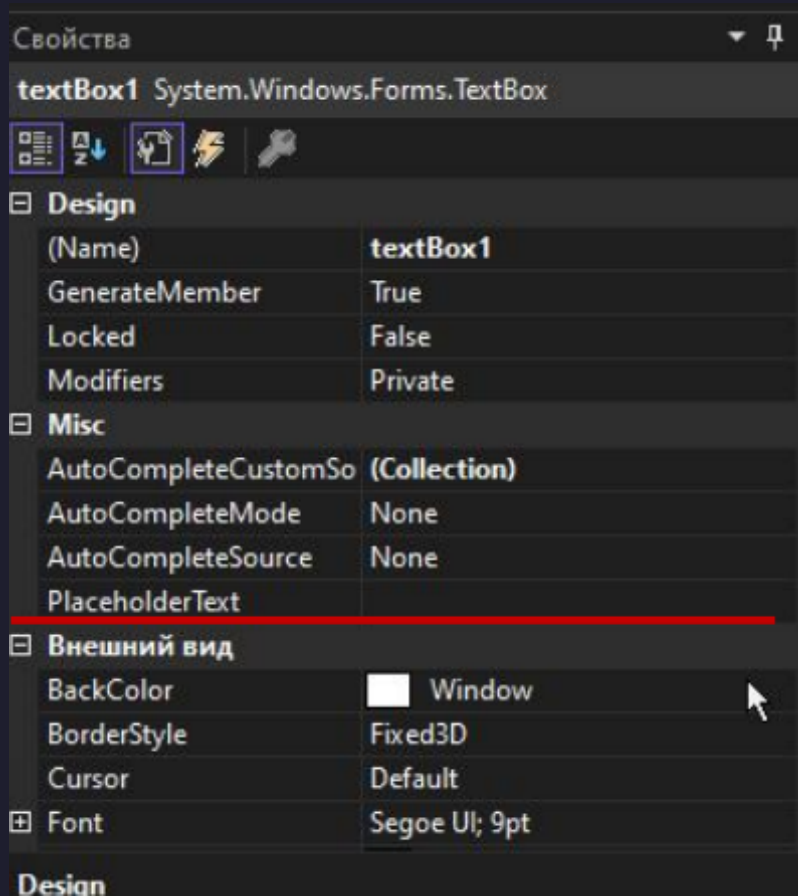


Рисунок 4.4 – Свойство Placeholder

Также необходимо настроить внешний вид элемента TextBox. Для того чтобы изменить высоту TextBox, необходимо изменить значение параметра Multiline на True (Рисунок 4.5).

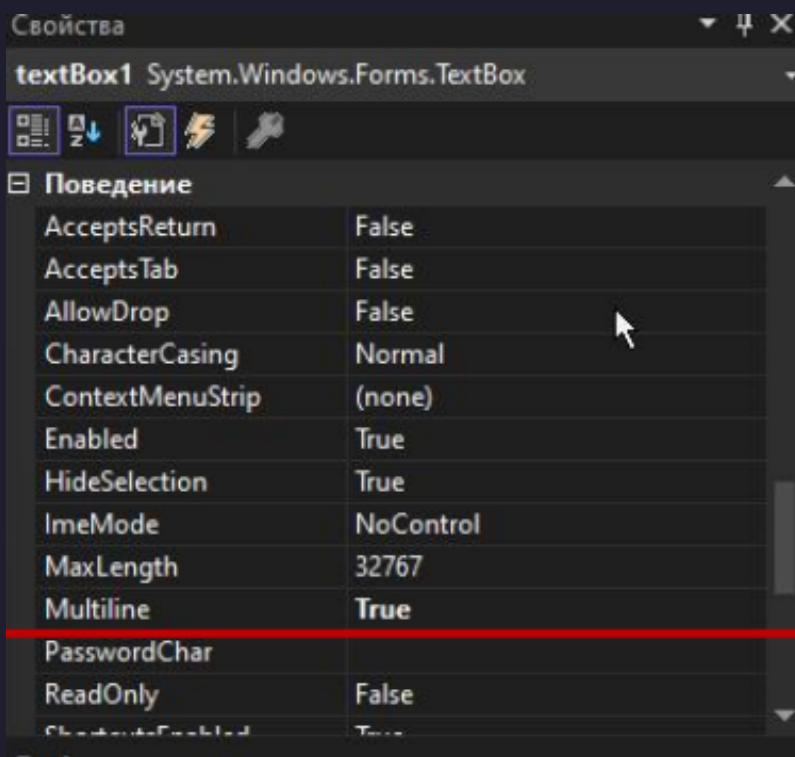


Рисунок 4.5 – Свойство Multiline

Для изменения фонового цвета используется свойство `BackColor`, а для изменения цвета текста – `ForeColor` (Рисунок 4.6).

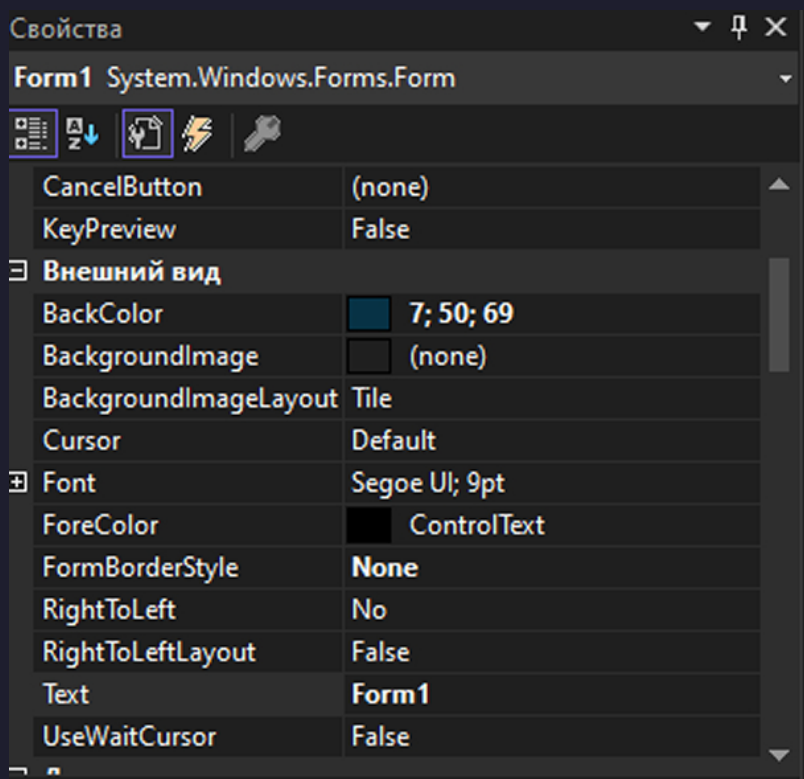


Рисунок 4.6 – Свойства для изменения цвета

Применив необходимые изменения, получим форму, изображенную на рисунке 4.7.

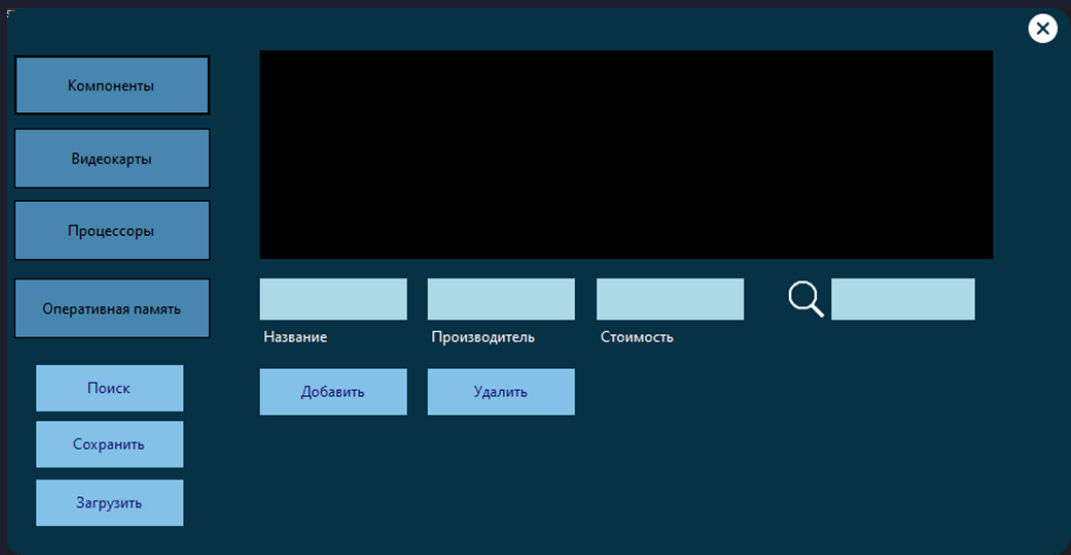


Рисунок 4.7 – Пример стилизованной формы⁴

Важно учесть расположение форм, которые в данном случае находятся друг над другом. Расположение панелей на форме указано на рисунке 4.8.

⁴Стилизация интерфейса на ваше усмотрение ☺

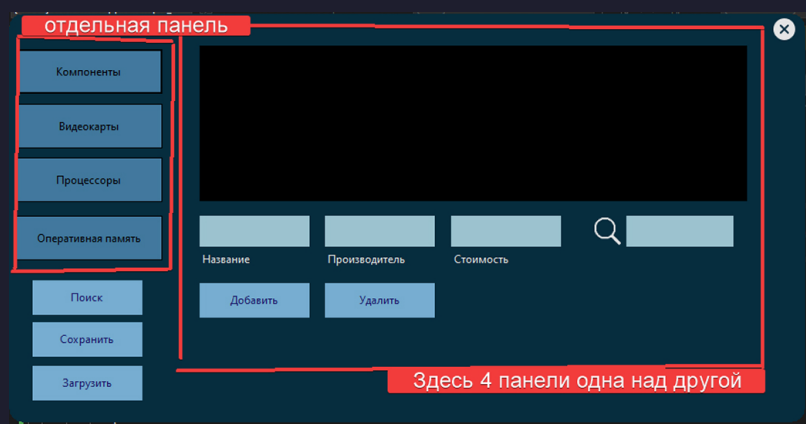


Рисунок 4.8 – Структура формы

Также для более детального рассмотрения и управления положением элементов на форме можно воспользоваться окном «Структура документа» (Вид → Другие окна → Структура документа), для этого нужно открыть файл с самой формой (Form.cs). Структура документа изображена на рисунке 4.9.

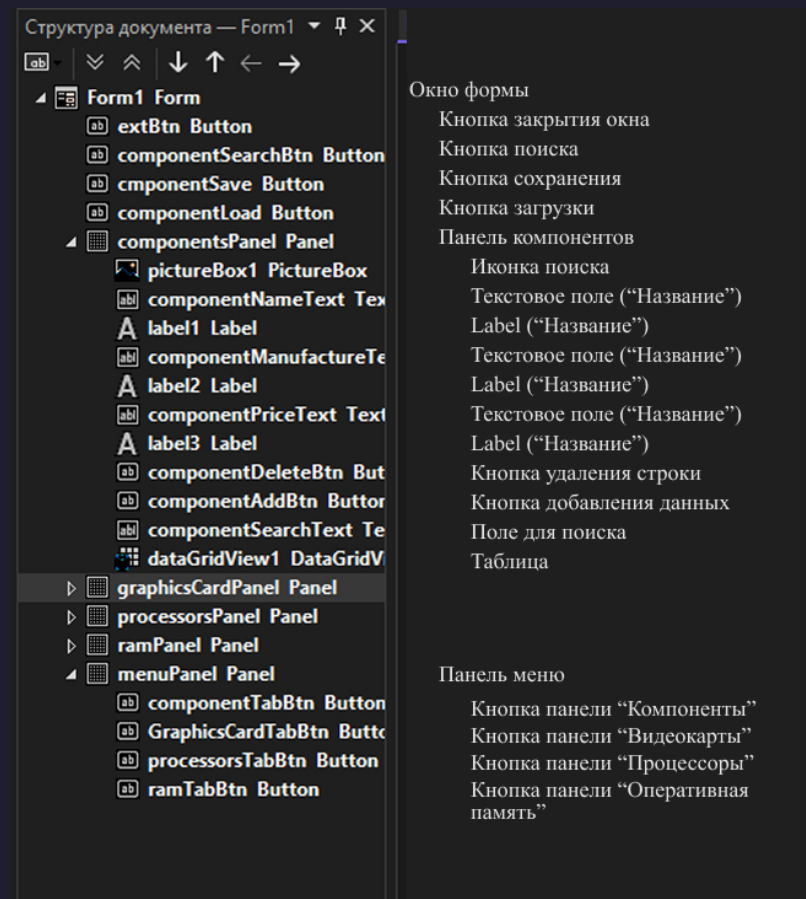


Рисунок 4.9 – Структура формы

5. Шаг 3. Добавление классов

Далее необходимо в Решении создать папку и добавить туда классы из лабораторной 1 (Рисунки 5.1).

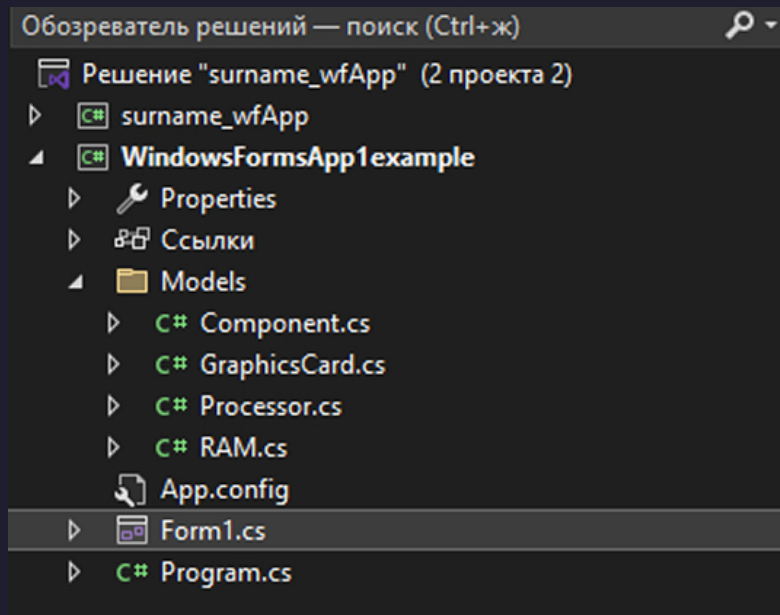


Рисунок 5.1 – Создание классов для работы с данными в форме

В каждом классе необходимо наличие 3 свойств, конструктора без параметров (нужен для сериализации) и конструктора с параметрами (Листинг 5.1).

```
9 namespace surname_wfApp.Models
10 {
11     public class Component
12     {
13         private List<Component> components = new List<Component>();
14
15         public string Name { get; set; }
16         public string Manufacturer { get; set; }
17         public decimal Price { get; set; }
18
19         // Конструктор по умолчанию
20         public Component() { }
21
22         public Component(string name, string manufacturer, decimal price)
23         {
24             Name = name; // Устанавливаем название
25             Manufacturer = manufacturer; // Устанавливаем производителя
26             Price = price; // Устанавливаем цену
27         }
28     }
29 }
```

Листинг 5.1 – Пример структуры класса

6. Шаг 4. Реализация добавления данных в таблицу

После добавления классов необходимо настроить обработчики событий. В WinForms у элементов управления (кнопок, чекбоксов, текстовых полей и т. д.) есть события (events), которые вызываются при определенных действиях пользователя. Например, клик по кнопке является одним из событий и при настройке появляется в окне свойств (Рисунок 6.1).

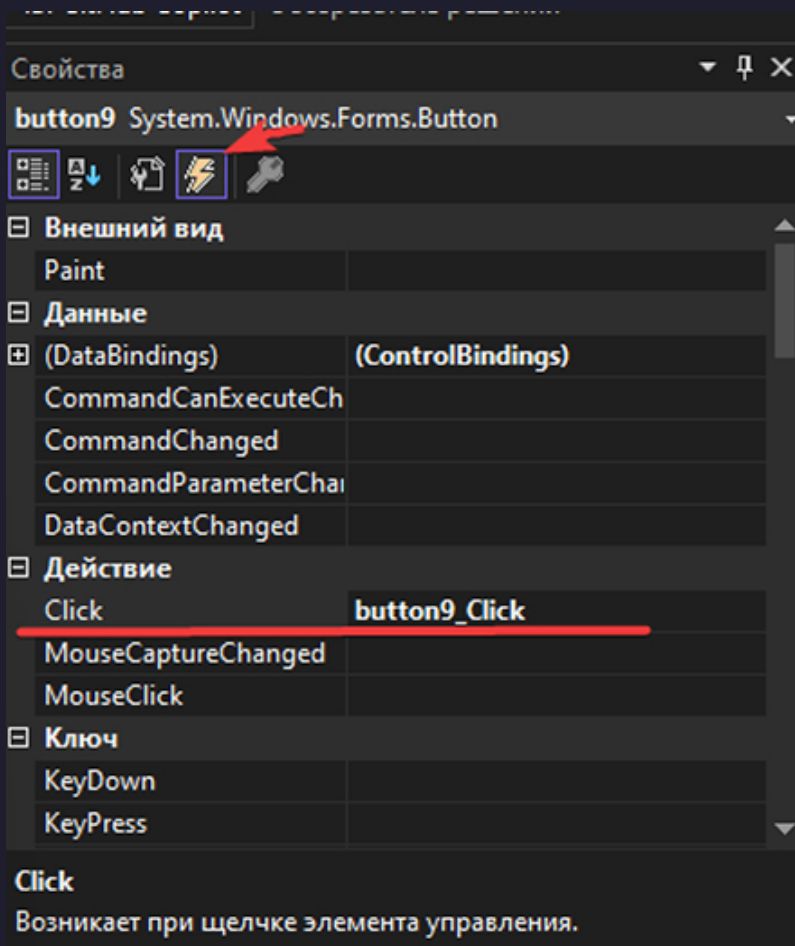


Рисунок 6.1 – Добавление событий

Если настраиваемое событие не обрабатывается, необходимо проверить панель свойств на наличие события в данном разделе.

Далее необходимо добавить событие нажатия на кнопку добавления данных в таблицу. Пример метода для добавления данных в форму представлен в листинге 6.1.

```
165 // Список компонентов (например, процессоры, видеокарты и т.д.)
166 private List<Component> componentList = new List<Component>();
167
168 private void componentAddBtn_Click_1(object sender, EventArgs e)
169 {
```



```
170     dataGridView1.AutoGenerateColumns = true;
171
172     // Проверяем заполненность полей
173     if (string.IsNullOrEmpty(componentNameText.Text) ||
174         string.IsNullOrEmpty(componentManufactureText.Text) ||
175         string.IsNullOrEmpty(componentPriceText.Text))
176     {
177         MessageBox.Show("Заполните все поля!", "Ошибка", MessageBoxButtons.OK,
178             ↪ MessageBoxIcon.Warning);
179         return;
180     }
181
182     // Преобразуем строковые значения в необходимые числовые типы
183     if (!decimal.TryParse(componentPriceText.Text, out decimal price))
184     {
185         MessageBox.Show("Введите корректное значение цены!", "Ошибка",
186             ↪ MessageBoxButtons.OK, MessageBoxIcon.Warning);
187         return;
188     }
189
190     // Создаем новый объект Component и добавляем его в List
191     Component newComponent = new Component(componentNameText.Text,
192         ↪ componentManufactureText.Text, price);
193     componentList.Add(newComponent);
194
195     // Очистка текстовых полей
196     componentNameText.Clear();
197     componentManufactureText.Clear();
198     componentPriceText.Clear();
199
200     // Обновляем DataGridView
201     filteredComponentList = null; // Сбрасываем фильтр после добавления
202     UpdateDataGrid();
203 }
```

Листинг 6.1 – Метод добавления данных в форму

Также необходимо после добавления данных обновить данные в таблице (DataGridView) (Листинг 6.2).

```
234     private void UpdateDataGrid()
235     {
236         // Переменные для хранения активного DataGridView и списка данных
237         DataGridView activeGridView = null;
238         IEnumerable<object> activeList = null;
239
240         // Определяем, какой DataGridView и список данных использовать
241         if (componentsPanel.Visible) // Панель компонентов
242         {
```

```
243         activeGridView = dataGridView1;
244         activeList = filteredComponentList ?? componentList;
245         // Используем отфильтрованный список, если он существует
246     }
247     else if (graphicsCardPanel.Visible) // Панель видеокарт
248     {
249         activeGridView = dataGridView2;
250         activeList = filteredGraphicsCardList ?? graphicsCardList;
251     }
252     else if (processorsPanel.Visible) // Панель процессоров
253     {
254         activeGridView = dataGridView4;
255         activeList = filteredProcessorsList ?? processorsList;
256     }
257
258     // Если активный DataGridView и список определены
259     if (activeGridView != null && activeList != null)
260     {
261         // Сбрасываем привязку данных
262         activeGridView.DataSource = null;
263
264         // Присваиваем обновленный список данных DataGridView
265         activeGridView.DataSource = activeList.ToList(); // Преобразуем в список, если это
        ↳ не List<T>
266     }
267     // Очистить все строки, чтобы избежать дублирования данных
268     dataGridView4.DataSource = null;
269
270     // Добавляем новые строки вручную для каждого объекта в списке
271     foreach (var processor in processorsList)
272     {
273         // Добавляем строку и указываем значения для каждого столбца
274         dataGridView4.Rows.Add(processor.Name, processor.Manufacturer,
275             processor.Price, processor.Frequency, processor.Cores);
276     }
277 }
```

Листинг 6.2 – Метод обновления DataGridView

Можно также добавить событие удаления элемента (строки) из таблицы (Листинг 6.3).

```
346 private void componentDeleteBtn_Click(object sender, EventArgs e)
347 {
348     if (dataGridView1.SelectedRows.Count == 0)
349     {
350         MessageBox.Show("Выберите элемент для удаления!", "Ошибка", MessageBoxButtons.OK,
        ↳ MessageBoxIcon.Warning);
351         return;
352     }
```

```
353
354     // Получаем выбранную строку из DataGridView
355     DataGridViewRow selectedRow = dataGridView1.SelectedRows[0];
356     // Получаем объект Component из DataSource строки
357     Component selectedComponent = (Component)selectedRow.DataBoundItem;
358
359     // Удаляем объект из основного списка
360     componentList.Remove(selectedComponent);
361     filteredComponentList = null; // Сбрасываем фильтр
362     UpdateDataGridView(); // Обновляем таблицу
363 }
```

Листинг 6.3 – Метод удаления строки из DataGridView

Для добавления данных в заранее созданные поля, необходимо связать название поля со свойствами создаваемого объекта (Листинг 6.4).

```
146     private void Form1_Load(object sender, EventArgs e)
147     {
148         // Стилизация GroupBox
149         ChangeControlsForeColor(groupBoxGC, Color.White);
150         groupBox12.ForeColor = this.BackColor; // Цвет рамки в цвет формы
151         groupBoxGC.ForeColor = this.BackColor; // Цвет рамки в цвет формы
152
153         //связывают кнопки сохранения/загрузки с DataGridView, к которому они относятся.
154         componentSave.Tag = dataGridView1;
155         componentLoad.Tag = dataGridView1;
156
157         // Устанавливаем привязку столбцов к свойствам объекта Processor
158         dataGridView4.Columns["Name"].DataPropertyName = "Name";
159         dataGridView4.Columns["Manufacturer"].DataPropertyName = "Manufacturer";
160         dataGridView4.Columns["Price"].DataPropertyName = "Price";
161         dataGridView4.Columns["Frequency"].DataPropertyName = "Frequency";
162         dataGridView4.Columns["Cores"].DataPropertyName = "Cores";
163     }
```

Листинг 6.4 – Добавление данных в столбцы DataGridView без автогенерации

Важно при работе с панелями динамически скрывать и делать их видимыми при выборе соответствующей вкладки. Для этого необходимо создать метод, который будет менять свойство `Visible` у соответствующей панели при нажатии на кнопку (Листинг 6.5).

```
100     private void HideAllPanels()
101     {
102         componentsPanel.Visible = false;
103         graphicsCardPanel.Visible = false;
```

```
104         processorsPanel.Visible = false;
105         ramPanel.Visible = false;
106     }
107
108     //Кнопки переключения вкладок
109     private void componentTabBtn_Click(object sender, EventArgs e)
110     {
111         HideAllPanels();
112         componentsPanel.Visible = true;
113     }
114
115     private void GraphicsCardTabBtn_Click(object sender, EventArgs e)
116     {
117         HideAllPanels();
118         graphicsCardPanel.Visible = true;
119     }
120
121     private void ramTabBtn_Click(object sender, EventArgs e)
122     {
123         HideAllPanels();
124         ramPanel.Visible = true;
125     }
126
127     private void processorsTabBtn_Click(object sender, EventArgs e)
128     {
129         HideAllPanels();
130         processorsPanel.Visible = true;
131     }
```

Листинг 6.5 – Метод скрытия панелей

7. Шаг 5. Реализация фильтрации (поиска) данных 🔍

Для поиска нужных данных в таблице используется поиск по подстроке в каждом столбце (листинг 7.1).

```

280 private void componentSearchBtn_Click(object sender, EventArgs e)
281 {
282     string query = string.Empty; // Объявляем переменную для хранения поискового запроса
283
284     // Определяем активную панель и берём текст из соответствующего поля ввода
285     if (componentsPanel.Visible) // Если активна панель компонентов
286         query = componentSearchText.Text.Trim().ToLower(); // Получаем текст из поля поиска компонентов,
287                                     // убираем пробелы и приводим к нижнему регистру
288     else if (graphicsCardPanel.Visible) // Если активна панель видеокарт
289         query = GCSearchText.Text.Trim().ToLower(); // Получаем текст из поля поиска видеокарт,
290                                     // убираем пробелы и приводим к нижнему регистру
291     else if (processorsPanel.Visible) // Если активна панель процессоров
292         query = processorSearchText.Text.Trim().ToLower(); // Получаем текст из поля поиска процессоров,
293                                     // убираем пробелы и приводим к нижнему регистру
294     else // Если ни одна панель не активна
295         return; // Завершаем выполнение метода
296
297     // Проверяем, пустой ли запрос
298     if (string.IsNullOrEmpty(query)) // Если запрос пустой
299     {
300         filteredComponentList = null; // Сбрасываем фильтр для компонентов
301         filteredGraphicsCardList = null; // Сбрасываем фильтр для видеокарт
302         filteredProcessorsList = null; // Сбрасываем фильтр для процессоров
303         UpdateDataGrid(); // Обновляем таблицу для отображения полного списка
304         return; // Завершаем выполнение метода
305     }
306
307     // Выполняем фильтрацию в зависимости от активной панели
308     if (componentsPanel.Visible) // Если активна панель компонентов
309     {
310         filteredComponentList = componentList.Where(comp => // Фильтруем список компонентов
311             comp.Name.ToLower().Contains(query) || // Поиск по имени (без учёта регистра)
312             comp.Manufacturer.ToLower().Contains(query) || // Поиск по производителю (без учёта регистра)
313             comp.Price.ToString().Contains(query) // Поиск по цене (как строке)
314         ).ToList(); // Преобразуем результат в список
315     }
316     else if (graphicsCardPanel.Visible) // Если активна панель видеокарт
317     {
318
319         filteredGraphicsCardList = graphicsCardList.Where(gc => // Фильтруем список видеокарт
320             gc.Name.ToLower().Contains(query) || // Поиск по имени (без учёта регистра)
321             gc.Manufacturer.ToLower().Contains(query) || // Поиск по производителю (без учёта регистра)
322             gc.Price.ToString().Contains(query) || // Поиск по цене (как строке)
323             gc.MemorySize.ToString().Contains(query) || // Поиск по объёму памяти (как строке)
324             gc.Type.ToLower().Contains(query) || // Поиск по типу памяти (без учёта регистра)
325             gc.BusWidth.ToString().Contains(query) // Поиск по ширине шины (как строке)
326         ).ToList(); // Преобразуем результат в список
327
328     }
329     else if (processorsPanel.Visible) // Если активна панель процессоров
330     {
331         filteredProcessorsList = processorsList.Where(proc => // Фильтруем список процессоров
332             proc.Name.ToLower().Contains(query) || // Поиск по имени (без учёта регистра)
333             proc.Manufacturer.ToLower().Contains(query) || // Поиск по производителю (без учёта регистра)
334             proc.Price.ToString().Contains(query) || // Поиск по цене (как строке)
335             proc.Frequency.ToString().Contains(query) || // Поиск по частоте (как строке)
336             proc.Cores.ToString().Contains(query) // Поиск по количеству ядер (как строке)
337         ).ToList(); // Преобразуем результат в список
338     }
339
340     UpdateDataGrid(); // Обновляем таблицу для отображения отфильтрованных данных
341 }

```

Листинг 7.1 – Метод фильтрации (поиска) данных в таблице

8. Шаг 6. Сериализация, десериализация классов. Сохранение в XML

Сериализация – это процесс преобразования объекта в поток байтов для его сохранения в файле, передаче по сети или хранении в базе данных. Впоследствии этот объект можно восстановить (десериализовать).

Типы сериализации в C#

В C# есть несколько типов сериализации:

- Binary
- XML
- JSON
- CSV⁵
- ...

В данном случае будет рассмотрена XML-сериализация. В C# для работы с XML используется класс `XmlSerializer` из пространства имен `System.Xml.Serialization`.

Пример метода сериализации с использованием `List` приведен в листинге 8.1.

```
531 private void SaveDataGridView(DataGridView dgv)
532 {
533     SaveFileDialog saveFileDialog = new SaveFileDialog(); // Создаём объект SaveFileDialog для выбора пользователем
                    ↳ пути и имени файла
534     saveFileDialog.Filter = "XML Files (*.xml)|*.xml|All Files (*.*)|*.*"; // Устанавливаем фильтр для отображения
535                                     // XML-файлов по умолчанию и опции "все
                                    ↳ файлы"
536
537     if (saveFileDialog.ShowDialog() == DialogResult.OK) // Открываем диалог сохранения и проверяем, выбрал ли
                    ↳ пользователь файл (нажал "OK")
538     {
539         string filePath = saveFileDialog.FileName; // Получаем полный путь к файлу, выбранному пользователем
                    ↳ (например, "C:\data.xml")
540
541         if (dgv == null) // Проверяем, был ли передан действительный DataGridView (не null)
542         {
543             MessageBox.Show("Нет доступных DataGridView для сохранения."); // Если DataGridView не передан,
                    ↳ показываем сообщение об ошибке
544             return; // Завершаем выполнение метода
545         }
546
547         // Выбираем, какой список сериализовать в зависимости от переданного DataGridView
548         if (dgv == dataGridView1) // Если передан DataGridView для компонентов (dataGridView1)
549         {
550             if (componentList.Count == 0) // Проверяем, содержит ли список componentList данные
```

⁵Это простой текстовый формат для хранения табличных данных, где каждая строка файла представляет одну строку таблицы. Значения в каждой строке разделяются определенным символом, чаще всего запятой.

```
551     {
552         MessageBox.Show("Нет данных для сохранения."); // Если список пуст, показываем сообщение
553         return; // Завершаем выполнение метода
554     }
555
556     XmlSerializer serializer = new XmlSerializer(typeof(List<Component>)); // Создаём XmlSerializer для
    ↪ сериализации
557
558     // списка List<Component> в XML
559     using (TextWriter writer = new StreamWriter(filePath)) // Создаём TextWriter (StreamWriter) для записи
    ↪ данных в файл
560     {
561         serializer.Serialize(writer, componentList); // Сериализуем componentList в XML и записываем в файл
562     } // Автоматически закрываем writer благодаря using
563     MessageBox.Show("Данные успешно сохранены."); // Сообщаем пользователю об успешном сохранении
564 }
565 else if (dgv == dataGridView2) // Если передан DataGridView для видеокарт (dataGridView2)
566 {
567     if (graphicsCardList.Count == 0) // Проверяем, содержит ли список graphicsCardList данные
568     {
569         MessageBox.Show("Нет данных для сохранения."); // Если список пуст, показываем сообщение
570         return; // Завершаем выполнение
571     }
572     Debug.WriteLine($"Graphics cards count: {graphicsCardList.Count}");
573
574     XmlSerializer serializer = new XmlSerializer(typeof(List<GraphicsCard>)); // Создаём XmlSerializer для
    ↪ сериализации
575
576     // списка List<GraphicsCard> в
    ↪ XML
577     using (TextWriter writer = new StreamWriter(filePath)) // Создаём TextWriter для записи данных в файл
578     {
579         serializer.Serialize(writer, graphicsCardList); // Сериализуем graphicsCardList в XML и записываем в
    ↪ файл
580     } // Закрываем writer
581     MessageBox.Show("Данные успешно сохранены."); // Сообщаем об успешном сохранении
582 }
583 else if (dgv == dataGridView4) // Если передан DataGridView для процессоров (dataGridView4)
584 {
585     if (processorsList.Count == 0) // Проверяем, содержит ли список processorsList данные
586     {
587         MessageBox.Show("Нет данных для сохранения."); // Если список пуст, показываем сообщение
588         return; // Завершаем выполнение
589     }
590
591     XmlSerializer serializer = new XmlSerializer(typeof(List<Processor>)); // Создаём XmlSerializer для
    ↪ сериализации
592
593     // списка List<Processor> в XML
594     using (TextWriter writer = new StreamWriter(filePath)) // Создаём TextWriter для записи данных в файл
595     {
596         serializer.Serialize(writer, processorsList); // Сериализуем processorsList в XML и записываем в файл
597     } // Закрываем writer
598     MessageBox.Show("Данные успешно сохранены."); // Сообщаем об успешном сохранении
599 }
600 }
```

Листинг 8.1 – Пример сериализации данных с помощью DataTable

Объяснение ключевых моментов:

- **SaveFileDialog:** Используется для взаимодействия с пользователем, чтобы выбрать место и имя файла. Filter ограничивает выбор XML-файлами, но позволяет выбрать «все файлы» (*.*)).
- **XmlSerializer:** Инструмент для преобразования объектов C# (в данном случае списков List<T>) в XML-формат и обратно. Он требует, чтобы классы (Component, GraphicsCard, Processor) имели публичные свойства и пустой конструктор.
- **using (TextWriter ...):** Обеспечивает корректное закрытие файла после записи, даже если произойдёт ошибка.
- **Проверка на пустой список:** Перед сериализацией проверяется Count, чтобы не создавать пустой XML-файл.
- **Условные ветви (if-else):** Код определяет, какой список сериализовать, основываясь на активном DataGridView. Это позволяет сохранять данные только для текущей вкладки.

Также можно использовать сериализацию данных с помощью DataTable, не используя списки List (Листинг 8.2).

```
1 private void SaveDataGridView(DataGridView dgv)
2 {
3     // Создаем и настраиваем диалог сохранения файла
4     using (SaveFileDialog saveFileDialog = new SaveFileDialog { Filter = "XML файлы|*.xml", Title
5         ↵ = "Сохранить данные" })
6     {
7         // Проверяем, нажал ли пользователь кнопку "Сохранить"
8         if (saveFileDialog.ShowDialog() == DialogResult.OK)
9         {
10             // Создаем DataTable с именем "MyTable"
11             DataTable dt = new DataTable("MyTable");
12
13             // Добавляем в DataTable колонки из DataGridView
14             foreach (DataGridViewColumn column in dgv.Columns)
15             {
16                 dt.Columns.Add(column.Name);
17             }
18
19             // Перебираем все строки DataGridView
20             foreach (DataGridViewRow row in dgv.Rows)
21             {
22                 // Пропускаем строку, если она новая (пустая)
23                 if (!row.IsNewRow)
24                 {
25                     // Создаем новую строку для DataTable
26                     DataRow dr = dt.NewRow();
```



```
27         // Заполняем строку данными из ячеек DataGridView
28         for (int i = 0; i < dgv.Columns.Count; i++)
29         {
30             dr[i] = row.Cells[i].Value ?? DBNull.Value; // Если значение null, заменяем
                 ↳ его на DBNull.Value
31         }
32
33         // Добавляем заполненную строку в DataTable
34         dt.Rows.Add(dr);
35     }
36 }
37
38 // Сохраняем DataTable в XML-файл с указанием схемы
39 dt.WriteXml(saveFileDialog.FileName, XmlWriteMode.WriteSchema);
40
41 // Выводим сообщение об успешном сохранении данных
42 MessageBox.Show("Данные сохранены!", "Успех", MessageBoxButtons.OK,
                 ↳ MessageBoxIcon.Information);
43 }
44 }
45 }
```

Листинг 8.2 – Пример сериализации данных с помощью DataTable

```
467 private void LoadDataGridView(DataGridView dgv) // Метод для загрузки данных из XML-файла в
                 ↳ список,
468
469                                     // привязанный к переданному DataGridView
470 {
471     OpenFileDialog openFileDialog = new OpenFileDialog(); // Создаём диалог для выбора
                 ↳ файла пользователем
472     openFileDialog.Filter = "XML Files (*.xml)|*.xml|All Files (*.*)|*.*"; // Устанавливаем
                 ↳ фильтр:
473
474                                     // XML-файлы по
475                                     ↳ умолчанию, с
476                                     ↳ опцией "все
477                                     ↳ файлы"
478
479     if (openFileDialog.ShowDialog() == DialogResult.OK) // Открываем диалог и проверяем,
                 ↳ выбрал ли пользователь файл
480     {
481         string filePath = openFileDialog.FileName; // Получаем путь к выбранному файлу
                 ↳ (например, "C:\data.xml")
482
483         if (dgv == null) // Проверяем, передан ли действительный DataGridView
484         {
485             MessageBox.Show("Передан некорректный DataGridView для загрузки данных."); //
                 ↳ Если dgv null, показываем ошибку
486             return; // Завершаем выполнение
487         }
488     }
489 }
```

```
482     }
483
484     // Загружаем данные в соответствующий список в зависимости от переданного
485     ↪ DataGridView
486     if (dgv == dataGridView1) // Если передан DataGridView для компонентов
487     {
488         XmlSerializer serializer = new XmlSerializer(typeof(List<Component>)); //
489         ↪ Создаём сериализатор для List<Component>
490         using (TextReader reader = new StreamReader(filePath)) // Открываем поток
491         ↪ чтения из файла
492         {
493             componentList = (List<Component>)serializer.Deserialize(reader); //
494             ↪ Десериализуем XML в componentList
495         }
496         filteredComponentList = null; // Сбрасываем фильтр для отображения полного
497         ↪ списка
498     }
499     else if (dgv == dataGridView2) // Если передан DataGridView для видеокарт
500     {
501         XmlSerializer serializer = new XmlSerializer(typeof(List<GraphicsCard>)); //
502         ↪ Создаём сериализатор для List<GraphicsCard>
503         using (TextReader reader = new StreamReader(filePath)) // Открываем поток чтения
504         {
505             graphicsCardList = (List<GraphicsCard>)serializer.Deserialize(reader); //
506             ↪ Десериализуем XML в graphicsCardList
507         }
508         filteredGraphicsCardList = null; // Сбрасываем фильтр
509     }
510     else if (dgv == dataGridView4) // Если передан DataGridView для процессоров
511     {
512         XmlSerializer serializer = new XmlSerializer(typeof(List<Processor>)); //
513         ↪ Создаём сериализатор для List<Processor>
514         using (TextReader reader = new StreamReader(filePath)) // Открываем поток чтения
515         {
516             processorsList = (List<Processor>)serializer.Deserialize(reader); //
517             ↪ Десериализуем XML в processorsList
518         }
519         filteredProcessorsList = null; // Сбрасываем фильтр
520     }
521     else // Если передан неизвестный DataGridView
522     {
523         MessageBox.Show("Неизвестный DataGridView для загрузки данных."); // Сообщаем
524         ↪ об ошибке
525         return; // Завершаем выполнение
526     }
527
528     UpdateDataGridView(); // Обновляем активный DataGridView с новыми данными
529     MessageBox.Show("Данные успешно загружены."); // Сообщаем об успехе
530 }
531 }
```

Листинг 8.3 – Пример метода десериализации⁶

Разбор метода десериализации.

```
467 private void LoadDataGridView(DataGridView dgv) // Метод для загрузки данных из XML-файла в
    ↳ список,
468 // привязанный к переданному DataGridView
```

Листинг 8.4 – Метод десериализации с параметром

Метод принимает параметр `dgv` типа `DataGridView`, чтобы знать, в какой `DataGridView` нужно загрузить данные (например, `dataGridView4` для процессоров, `dataGridView1` для компонентов или `dataGridView2` для видеокарт). Такой подход делает метод универсальным – он может работать с любым `DataGridView`, а не с конкретным. Это удобно, если у вас несколько таблиц (`dataGridView1`, `dataGridView2`, `dataGridView4`), и вы хотите переиспользовать код (Листинг 8.4).

```
470 OpenFileDialog openFileDialog = new OpenFileDialog(); // Создаём диалог для выбора
    ↳ файла пользователем
471 openFileDialog.Filter = "XML Files (*.xml)|*.xml|All Files (*.*)|*.*"; // Устанавливаем
    ↳ фильтр:
472 // XML-файлы по
    ↳ умолчанию, с
    ↳ опцией "все
    ↳ файлы"
```

Листинг 8.5 – Объект `OpenFileDialog`

Создается объект `OpenFileDialog` для выбора файла пользователем. Свойство `Filter` задаёт фильтр файлов, чтобы в диалоге отображались только XML-файлы по умолчанию, но с возможностью выбрать «все файлы». Фильтр упрощает пользователю выбор нужного файла, показывая только файлы с расширением `.xml`.

Формат фильтра `"XML Files (*.xml)|*.xml|All Files (*.*)|*.*"` – это стандартный синтаксис для `OpenFileDialog`. Первая часть (`XML Files (*.xml)`) – это описание, вторая (`*.xml`) – маска файлов. Аналогично для `"All Files"` (Листинг 8.5).

```
474 if (openFileDialog.ShowDialog() == DialogResult.OK) // Открываем диалог и проверяем,
    ↳ выбрал ли пользователь файл
475 {
```

⁶Десериализация – это процесс обратного преобразования, то есть восстановления объекта из сохраненного состояния.

```
476     string filePath = openFileDialog.FileName; // Получаем путь к выбранному файлу
      ↳ (например, "C:\data.xml")
477
478     if (dgv == null) // Проверяем, передан ли действительный DataGridView
479     {
480         MessageBox.Show("Передан некорректный DataGridView для загрузки данных."); //
      ↳ Если dgv null, показываем ошибку
481         return; // Завершаем выполнение
482     }
```

Листинг 8.6 – Выбор файла и проверка на корректную таблицу

Метод `ShowDialog()` (Листинг 8.6) открывает диалоговое окно выбора файла. Если пользователь выбрал файл и нажал «Открыть», возвращается `DialogResult.OK`. Если пользователь нажал «Отмена», метод завершится, ничего не делая.

Свойство `FileName` возвращает полный путь к выбранному файлу (например, `C:\data.xml`). Этот путь нужен для чтения данных из файла. Без пути не получится открыть файл для десериализации.

```
1  if (dgv == null)
2  {
3      MessageBox.Show("Передан некорректный DataGridView для загрузки данных.");
4      return;
5  }
```

Проверяется, что переданный `DataGridView` (`dgv`) не равен `null`. Если `dgv` равен `null`, дальнейшая работа с ним вызовет исключение `NullReferenceException`. Эта проверка защищает от таких ошибок.

```
485     if (dgv == dataGridView1) // Если передан DataGridView для компонентов
486     {
487         XmlSerializer serializer = new XmlSerializer(typeof(List<Component>)); //
      ↳ Создаём сериализатор для List<Component>
488         using (TextReader reader = new StreamReader(filePath)) // Открываем поток
      ↳ чтения из файла
489         {
490             componentList = (List<Component>)serializer.Deserialize(reader); //
      ↳ Десериализуем XML в componentList
491         }
492         filteredComponentList = null; // Сбрасываем фильтр для отображения полного
      ↳ списка
493     }
```

Листинг 8.7 – Сериализация XML

Далее рассмотрим объект `XmlSerializer`, позволяющий сохранять данные из таблицы в формате XML (Листинг 8.7).

Если переданный `DataGridView` – это `dataGridView1` (для компонентов), данные загружаются в `componentList`. Создаётся объект `XmlSerializer` для типа `List<Component>`. `XmlSerializer` знает, как преобразовать XML в список объектов `Component`; это указывает, что ожидается XML-файл, содержащий список объектов `Component`.

`StreamReader` открывает файл по пути `filePath` для чтения текста. Конструкция `using` гарантирует, что поток (`StreamReader`) будет закрыт после использования, даже если произойдёт исключение.

Метод `Deserialize` читает XML из потока и преобразует его в объект типа `List<Component>`. Приведение (`List<Component>`) необходимо, так как `Deserialize` возвращает объект типа `object`. Сбрасывается фильтр, чтобы после загрузки отображался полный список компонентов, а не отфильтрованный. Этот блок загружает данные из XML-файла в список `componentList`, который привязан к `dataGridView1`. Сброс фильтра (`filteredComponentList = null`) гарантирует, что пользователь увидит все загруженные данные.

Почему так: Использование `XmlSerializer` – это стандартный способ десериализации XML в .NET. Проверка `dgv == dataGridView1` позволяет методу работать с разными `DataGridView` и списками. Также важно связать события кнопок сохранения и загрузки с соответствующими таблицами (Листинг 8.8).

```
146     private void Form1_Load(object sender, EventArgs e)
147     {
148         // Стилизация GroupBox
149         ChangeControlsForeColor(groupBoxGC, Color.White);
150         groupBox12.ForeColor = this.BackColor; // Цвет рамки в цвет формы
151         groupBoxGC.ForeColor = this.BackColor; // Цвет рамки в цвет формы
152
153         //связывают кнопки сохранения/загрузки с DataGridView, к которому они относятся.
154         componentSave.Tag = dataGridView1;
155         componentLoad.Tag = dataGridView1;
156
157         // Устанавливаем привязку столбцов к свойствам объекта Processor
158         dataGridView4.Columns["Name"].DataPropertyName = "Name";
159         dataGridView4.Columns["Manufacturer"].DataPropertyName = "Manufacturer";
160         dataGridView4.Columns["Price"].DataPropertyName = "Price";
161         dataGridView4.Columns["Frequency"].DataPropertyName = "Frequency";
162         dataGridView4.Columns["Cores"].DataPropertyName = "Cores";
163     }
```

Листинг 8.8 – Связь событий и кнопок с помощью Tag

9. Дополнительная информация ?

1. Обязательно ли указывать атрибут Serializable?

Нет, атрибут [Serializable] не нужен, если используется DataSet / DataTable + XML для сохранения данных.

2. Почему Serializable не важен?

Атрибут [Serializable] требуется, если объекты сохраняются с помощью бинарной или JSON-сериализации (например, BinaryFormatter или JsonSerializer).

НО при использовании DataSet.WriteXml(filePath) не нужно указывать атрибут, так как:

- ✓ DataSet.WriteXml(filePath) сохраняет данные в XML без необходимости сериализации класса.
- ✓ Работает с таблицами (DataTable), а не с объектами конкретного List.

Когда [Serializable] был бы нужен:

- Если объекты List сохраняются в файл (например, JSON, Binary, SOAP).
- Если используется BinaryFormatter или JsonConvert.SerializeObject(graphicsCardList)

Если необходимо добавлять данные в уже созданные ячейки DataGridView, а не создавать новые строки, необходимо обновлять значения ячеек в существующих строках. Это можно сделать несколькими способами:

1. Обращение к ячейкам по индексу строки и столбца.

В случае, если в DataGridView уже присутствуют строки, можно обновлять данные так:

```
1 // Обновление данных в первой строке (индекс 0)
2 dataGridView1.Rows[0].Cells[0].Value = "Новое значение";
3 dataGridView1.Rows[0].Cells[1].Value = 123;
```

2. Работа с DataTable (если DataGridView привязан к нему).

Если DataGridView использует DataTable как источник данных:

```
1 // Получаем доступ к DataTable
2 DataTable dt = (DataTable)dataGridView1.DataSource; // Изменяем данные в нужной строке и
   ↳ столбце
3 dt.Rows[0]["НазваниеСтолбца"] = "Новое значение";
```

После этого DataGridView обновится автоматически.

3. Обновление данных через BindingList<T>.

Если DataGridView привязан к BindingList<T>, можно менять объект в списке:

```
1 // Предположим, у нас есть BindingList<MyObject> bindingList;  
2 // bindingList[0].SomeProperty = "Новое значение";  
3 dataGridView1.Refresh(); // Принудительно обновляем DataGridView
```

Важно:

- Убедитесь, что строки уже существуют перед обновлением (dataGridView1.Rows.Count).
- Если DataGridView связан с DataTable или BindingList<T>, изменения надо делать в источнике данных.

10. Стилизация интерфейса 🏠

После этого возможность перемещать форму с помощью мыши будет отключена. Чтобы вернуть возможность перемещения формы, необходимо добавить флаги для реализации перемещения формы, функции для перемещения формы и обработчики событий нажатия мыши. Также можно стилизовать форму, закруглив края с помощью функции `CreateRoundRectRgn`.

Флаги для реализации перемещения формы и функции для перемещения формы, а также функции для скругления краев формы представлены в листинге 10.1.

```

22 // Флаги для реализации перемещения формы
23 private bool dragging = false; // Состояние перетаскивания (активно/неактивно)
24 private Point startPoint = new Point(0, 0); // Начальные координаты клика мыши
25
26 // Импорт функции CreateRoundRectRgn для создания закругленных углов формы
27 [DllImport("Gdi32.dll", EntryPoint = "CreateRoundRectRgn")]
28 private static extern IntPtr CreateRoundRectRgn(
29     int nLeftRect, // X-координата левого верхнего угла
30     int nTopRect, // Y-координата левого верхнего угла
31     int nRightRect, // X-координата правого нижнего угла
32     int nBottomRect, // Y-координата правого нижнего угла
33     int nWidthEllipse, // Ширина эллипса для закругления углов
34     int nHeightEllipse // Высота эллипса для закругления углов
35 );
36
37 // Импорт функций для перетаскивания формы
38 [DllImport("user32.dll")]
39 private static extern void ReleaseCapture(); // Сброс захвата мыши
40
41 [DllImport("user32.dll")]
42 private static extern void SendMessage(IntPtr hWnd, int msg, int wp, int lp);
43
44 private const int WM_NCLBUTTONDOWN = 0xA1; // Сообщение о нажатии кнопки мыши в
    ↳ незакрашенной области окна
45 private const int HTCAPTION = 0x2; // Идентификатор области заголовка окна

```

Листинг 10.1 – Функции реализации перемещения формы

Обработчики нажатия мыши приведены в листинге 10.2.

```

47 // Обработчик события нажатия мыши на форме
48 private void Form1_MouseDown(object sender, MouseEventArgs e)
49 {
50     // Включаем режим перетаскивания формы
51     dragging = true;
52     // Запоминаем начальные координаты клика относительно формы
53     startPoint = new Point(e.X, e.Y);

```



```

54     }
55
56     // Обработчик события движения мыши над формой
57     private void Form1_MouseMove(object sender, MouseEventArgs e)
58     {
59         // Проверяем, активировано ли перетаскивание
60         if (dragging)
61         {
62             // Преобразуем текущие координаты мыши в экранные координаты
63             Point newPosition = PointToScreen(new Point(e.X, e.Y));
64             // Обновляем позицию формы с учетом начальной точки клика
65             // Это позволяет перемещать форму за точку, где был совершен клик
66             this.Location = new Point(
67                 newPosition.X - startPoint.X,
68                 newPosition.Y - startPoint.Y
69             );
70         }
71     }
72
73     // Обработчик события отпускания кнопки мыши
74     private void Form1_MouseUp(object sender, MouseEventArgs e)
75     {
76         // Выключаем режим перетаскивания
77         dragging = false;
78     }

```

Листинг 10.2 – Обработчики движения мыши

Также в примере применяется изменение цвета элементов Label, размещенных внутри GroupBox (Листинг 10.3).

```

134     //Стилизация: смена цвета label внутри groupbox
135     private void ChangeControlsForeColor(System.Windows.Forms.GroupBox groupBox, Color color)
136     {
137         foreach (Control control in groupBox.Controls)
138         {
139             if (!(control is System.Windows.Forms.TextBox) && !(control is
140                 ↪ System.Windows.Forms.Button)) // Исключаем TextBox и Button
141             {
142                 control.ForeColor = color;
143             }
144         }
145
146     private void Form1_Load(object sender, EventArgs e)
147     {
148         // Стилизация GroupBox
149         ChangeControlsForeColor(groupBoxGC, Color.White);
150         groupBox12.ForeColor = this.BackColor; // Цвет рамки в цвет формы

```

```
151     groupBoxGC.ForeColor = this.BackColor; // Цвет рамки в цвет формы
152
153     //связывают кнопки сохранения/загрузки с DataGridView, к которому они относятся.
154     componentSave.Tag = dataGridView1;
155     componentLoad.Tag = dataGridView1;
156
157     // Устанавливаем привязку столбцов к свойствам объекта Processor
158     dataGridView4.Columns["Name"].DataPropertyName = "Name";
159     dataGridView4.Columns["Manufacturer"].DataPropertyName = "Manufacturer";
160     dataGridView4.Columns["Price"].DataPropertyName = "Price";
161     dataGridView4.Columns["Frequency"].DataPropertyName = "Frequency";
162     dataGridView4.Columns["Cores"].DataPropertyName = "Cores";
163 }
```

Листинг 10.3 – Изменение цвета Label внутри GroupBox

11. Форматирование данных

Для того чтобы отформатировать данные в форме, например автоматически приводить стоимость к денежному формату (добавлять пробел, числа после запятой и знак валюты), необходимо добавить метод, который будет изменять содержимое соответствующего столбца (Рисунок 11.1, Листинг 11.1).

	Name	Manufacturer	Price	MemorySize	Type	BusWidth
▶	Gh12	China	120 000,00 ₺	16	GDDR6	128

Рисунок 11.1 – Форматирование столбца Price

```
202 private void FormatPriceCell(DataGridView dgv, DataGridViewCellFormattingEventArgs e)
203 {
204     // Проверяем, что индекс корректен
205     if (e.ColumnIndex < 0 || e.ColumnIndex >= dgv.Columns.Count)
206         return;
207
208     // Проверяем колонку "Price" и наличие значения
209     if (dgv.Columns[e.ColumnIndex].Name == "Price" && e.Value != null)
210     {
211         if (decimal.TryParse(e.Value.ToString(), out decimal price))
212         {
213             e.Value = price.ToString("N2") + " ₺";
214             e.FormattingApplied = true;
215         }
216     }
217 }
```

Листинг 11.1 – Метод форматирования строки со стоимостью

А затем вызывать этот метод у каждой таблицы (свойство `CellFormatting`) (Листинг 11.2).

```
219 private void dataGridView1_CellFormatting(object sender,
220     ↪ DataGridViewCellFormattingEventArgs e)
221 {
222     FormatPriceCell(dataGridView1, e);
223 }
224
225 private void dataGridView2_CellFormatting(object sender,
226     ↪ DataGridViewCellFormattingEventArgs e)
227 {
228     FormatPriceCell(dataGridView2, e);
229 }
```

```
228
229     private void dataGridView4_CellFormatting(object sender,
    ↪     DataGridViewCellFormattingEventArgs e)
230     {
231         FormatPriceCell(dataGridView4, e);
232     }
```

Листинг 11.2 – Вызов метода форматирования

12. Интерфейс приложения

Далее представлены примеры интерфейса приложения. Добавление данных в таблицу Components представлено на рисунке 12.1.

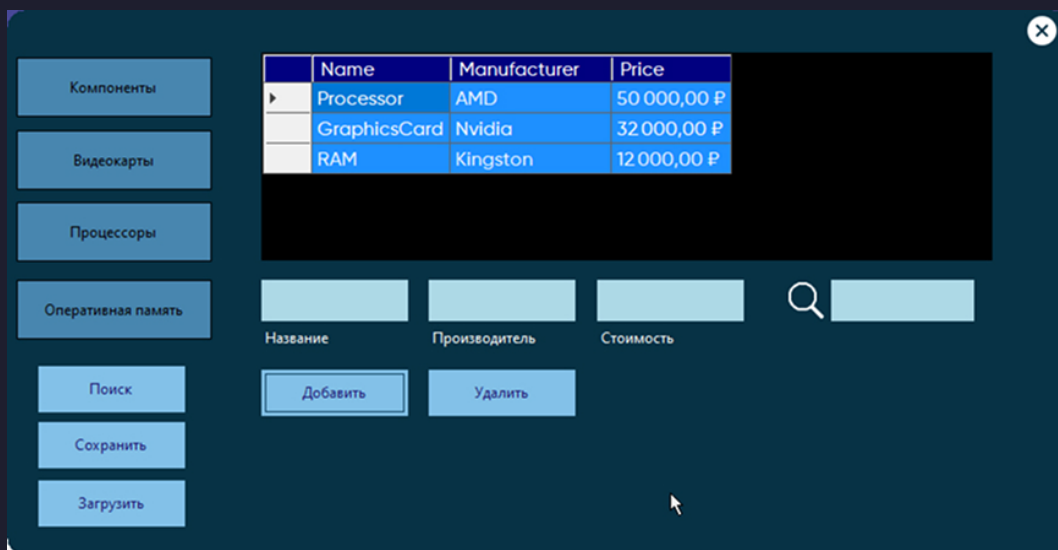


Рисунок 12.1 – Добавление данных в таблицу компонентов

Удаление осуществляется нажатием на соответствующую строку и затем нажатием кнопки «Удалить» (Рисунок 12.2).

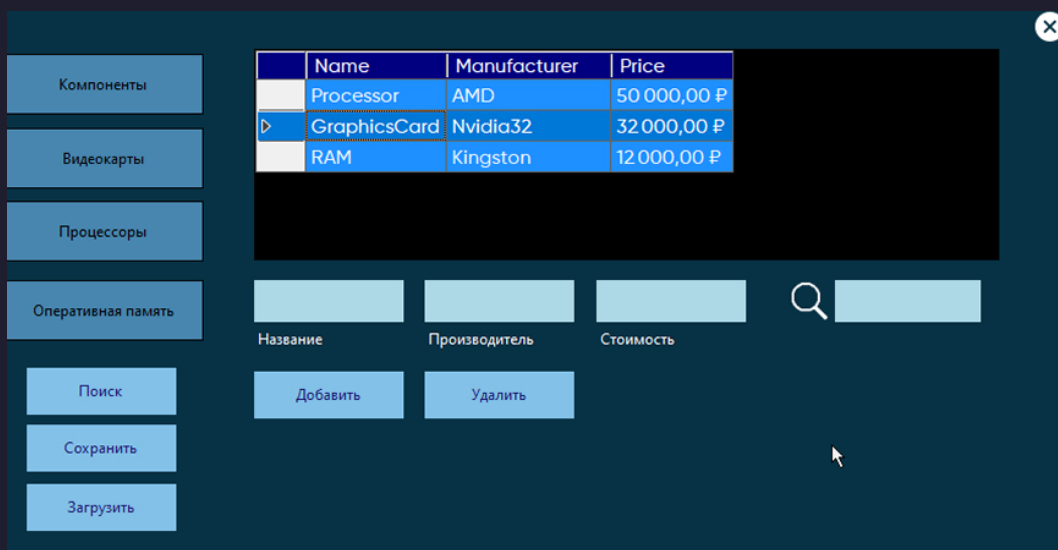


Рисунок 12.2 – Удаление данных из таблицы компонентов

Поиск осуществляется путем ввода данных в поле поиска и нажатием кнопки «Поиск». Поля поиска в данном случае уникальные для каждой панели (класса), но кнопка поиска одна (Рисунок 12.3).

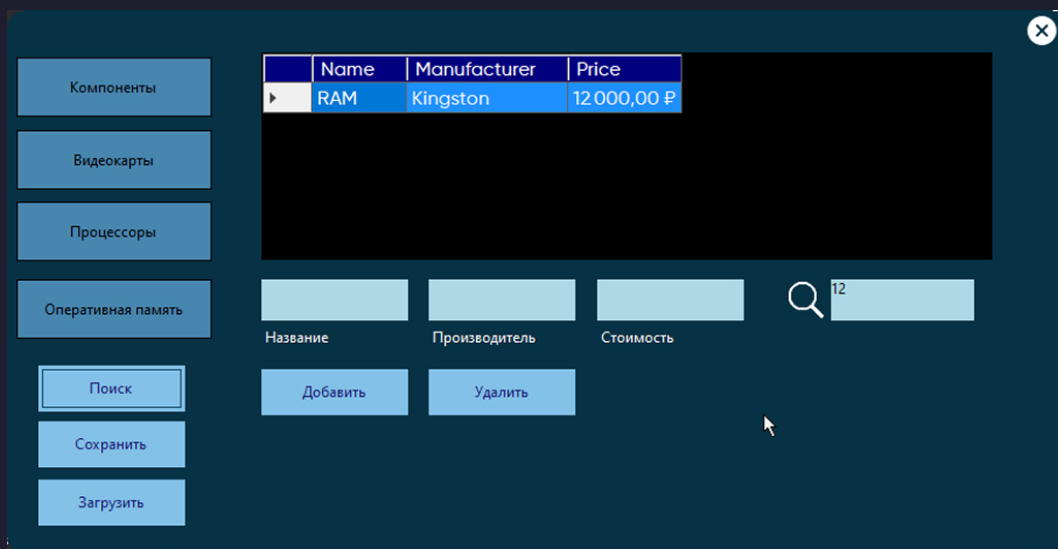


Рисунок 12.3 – Поиск данных в таблице компонентов

Поиск осуществляется путем ввода данных в поле поиска и нажатием кнопки «Поиск». Поля поиска в данном случае уникальные для каждой панели (класса), но кнопка поиска одна (Рисунок 12.4).

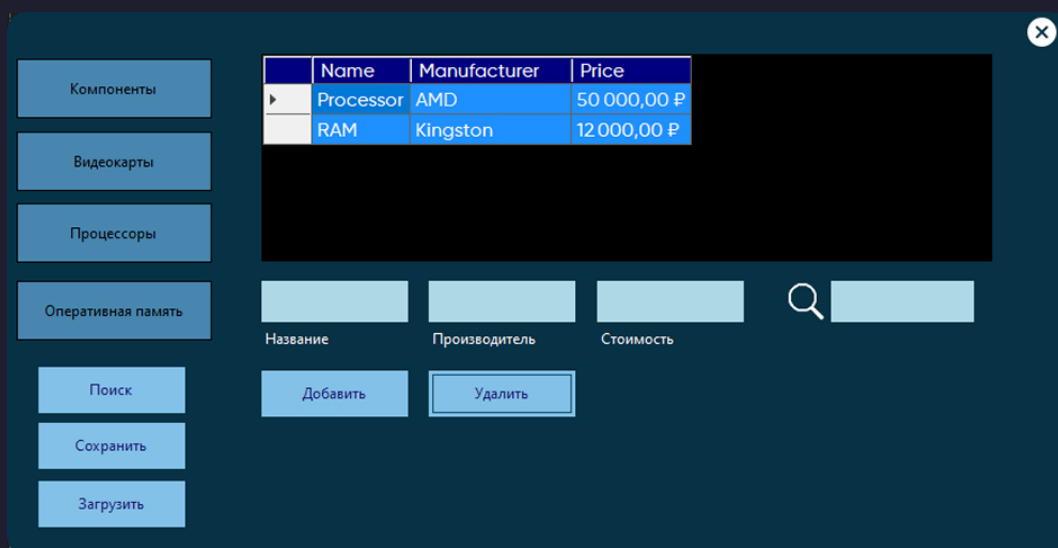


Рисунок 12.4 – Поиск данных в таблице компонентов (пример с результатом)

Также реализовано добавление данных в таблицы других классов (Рисунок 12.5).

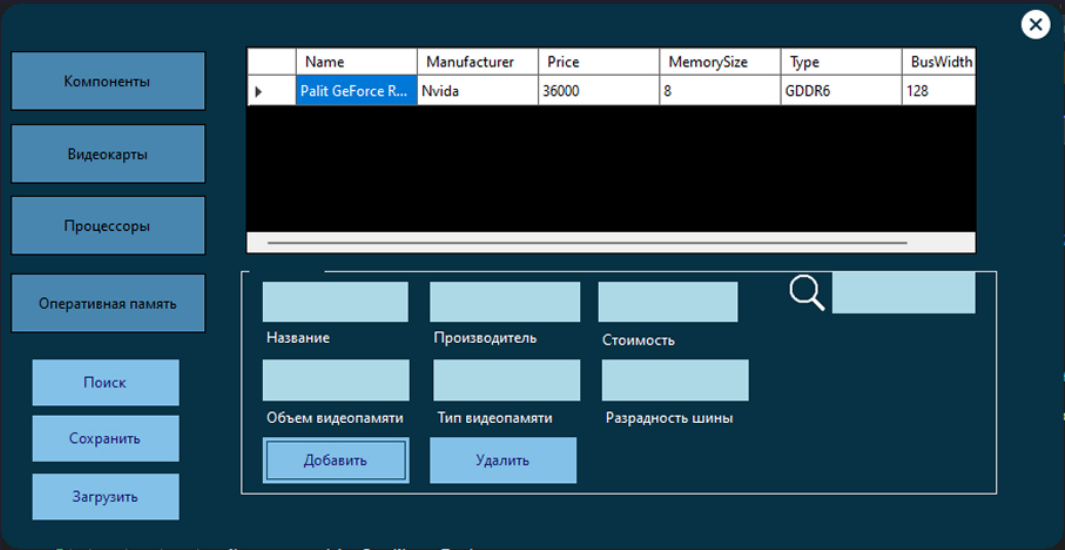


Рисунок 12.5 – Добавление данных в таблицу видеокарт